

Expense Tracker - Guide technique pour modifier le projet

Ce document explique les fichiers essentiels du projet. L'objectif est de te permettre d'ajouter tes propres modifications sans te perdre dans l'architecture.

Vue d'ensemble

Le projet est separe en deux applications:

- ? ``backend/``: API Django REST qui gere les utilisateurs, les depenses, les statistiques et les rapports PDF.
- ? ``frontend/``: interface React/Vite qui consomme l'API avec Axios.

Le principe important: chaque depense appartient a un utilisateur. Le backend filtre toujours les donnees avec `request.user`, donc un utilisateur ne voit pas les depenses d'un autre.

Backend Django

``backend/manage.py``

Point d'entree des commandes Django.

Tu l'utilises pour:

- ? lancer le serveur: ``python manage.py runserver``
- ? appliquer les migrations: ``python manage.py migrate``
- ? verifier la configuration: ``python manage.py check``
- ? ouvrir un shell Django: ``python manage.py shell``

``backend/requirements.txt``

Liste les dependances Python:

- ? Django
- ? Django REST Framework
- ? Simple JWT
- ? `django-cors-headers`
- ? `django-filter`
- ? ReportLab
- ? gunicorn et whitenoise pour le deploiement

Si tu ajoutes une librairie backend, elle doit etre ajoutez ici.

``backend/config/settings.py``

Configuration principale Django.

Parties importantes:

- ? ``INSTALLED_APPS``: active Django, DRF, JWT, CORS et les apps métier.
- ? ``DATABASES``: SQLite en developpement.
- ? ``REST_FRAMEWORK``: force l'authentification JWT par default.
- ? ``SIMPLE_JWT``: duree de vie des tokens.
- ? ``CORS_ALLOWED_ORIGINS``: autorise le frontend a appeler l'API.

Quand tu deployes, tu dois surtout modifier les variables:

- ? ``SECRET_KEY``
- ? ``DEBUG=False``
- ? ``ALLOWED_HOSTS``
- ? ``CORS_ALLOWED_ORIGINS``

``backend/config/urls.py``

Carte principale des routes API.

Routes importantes:

- ? ``/api/auth/register/``: inscription
- ? ``/api/auth/login/``: connexion JWT
- ? ``/api/auth/refresh/``: renouvellement token
- ? ``/api/auth/me/``: profil utilisateur connecte
- ? ``/api/expenses/``: CRUD dépenses
- ? ``/api/expenses/stats/``: statistiques dashboard
- ? ``/api/reports/``: resume de rapport
- ? ``/api/reports/pdf/``: export PDF

Si tu crees une nouvelle app backend, c'est souvent ici que tu connecteras ses URLs.

``backend/users/serializers.py``

Transforme les donnees utilisateur entre JSON et modele Django.

RegisterSerializer:

- ? valide l'email unique
- ? impose un mot de passe via Django
- ? hash le mot de passe avec ``set_password``

UserSerializer:

? retourne les informations publiques de l'utilisateur connecte.

``backend/users/views.py``

Contient les vues d'inscription et de profil.

? ``RegisterView``: accessible sans etre connecte.

? ``UserProfileView``: retourne ``request.user``.

Si tu veux ajouter un endpoint de modification de profil, c'est ici que tu peux commencer.

``backend/expenses/models.py``

Modele principal de l'application.

Expense contient:

? ``user``: proprietaire de la depense

? ``amount``: montant

? ``category``: categorie

? ``description``: description optionnelle

? ``date``: date

? ``payment_method``: mode de paiement

? ``created_at`` / ``updated_at``

Les categories et moyens de paiement sont definis avec TextChoices.

Si tu veux ajouter un champ comme merchant, location ou receipt_image, il faut:

1. modifier ce fichier;
2. creer une migration;
3. adapter le serializer;
4. adapter le formulaire React.

``backend/expenses/serializers.py``

Expose le modele Expense en JSON.

Il ajoute aussi:

? ``category_label``

? ``payment_method_label``

La validation du montant est ici: le montant doit etre superieur a zero.

``backend/expenses/views.py``

Fichier cle pour les depenses.

ExpenseViewSet gere:

- ? liste
- ? creation
- ? modification
- ? suppression
- ? recherche
- ? filtres
- ? tri
- ? statistiques

Point de securite important:

```
Expense.objects.filter(user=self.request.user)
```

Cette ligne garantit que l'utilisateur ne travaille que sur ses propres depenses.

La methode stats() calcule:

- ? total du jour
- ? total semaine
- ? total mois
- ? total general
- ? categorie dominante
- ? derniere depense
- ? donnees pour les graphiques

``backend/reports/views.py``

Gere les rapports:

- ? journalier
- ? hebdomadaire
- ? mensuel

La methode pdf() genere le fichier PDF avec ReportLab.

Si tu veux enrichir le PDF avec logo, couleurs, graphiques ou resume plus complet, ce fichier est le bon endroit.

``backend/expenses/migrations/``

Historique des changements de base de données.

Ne modifie pas une migration déjà appliquée sauf si tu sais exactement pourquoi. Pour un nouveau changement de modèle, crée une nouvelle migration.

Frontend React

``frontend/package.json``

Liste les scripts et dépendances frontend.

Scripts utiles:

? ``npm run dev``: serveur de développement

? ``npm run build``: build production

? ``npm run lint``: vérification qualité

Dépendances principales:

? React

? React Router

? Axios

? Recharts

? React Hook Form

? TailwindCSS

? React Hot Toast

``frontend/src/main.jsx``

Point d'entrée React.

Il monte l'application dans `#root` et ajoute:

? ``AuthProvider``

? ``Toaster``

Si tu veux ajouter un provider global comme Zustand, ThemeProvider ou QueryClient, c'est ici.

``frontend/src/App.jsx``

Charge le routeur React.

Le fichier reste volontairement simple.

``frontend/src/routes/router.jsx``

Carte des routes frontend.

Routes privees:

- ? `/`: dashboard
- ? `/expenses`: depenses
- ? `/reports`: rapports

Routes publiques:

- ? `/login`
- ? `/register`

errorElement affiche une page plus propre si une route plante.

`frontend/src/routes/ProtectedRoute.jsx`

Bloque les pages privees si l'utilisateur n'est pas connecte.

Si l'utilisateur n'a pas de session, il est redirige vers /login.

`frontend/src/routes/PublicRoute.jsx`

Evite qu'un utilisateur deja connecte retourne sur login/register.

`frontend/src/context/AuthContext.jsx`

Gere l'etat d'authentification.

Responsabilites:

- ? charger l'utilisateur connecte avec `/auth/me/`
- ? login
- ? register
- ? logout
- ? stocker les tokens JWT dans `localStorage`

Si tu veux changer la gestion auth, par exemple passer a des cookies HTTP-only, c'est un fichier central a revoir.

`frontend/src/services/api.js`

Configuration Axios globale.

Il ajoute automatiquement:

```
Authorization: Bearer <token>
```

Il tente aussi de rafraichir le token si l'API repond 401.

Si l'URL backend change, elle vient de:

```
VITE_API_URL=http://localhost:8000/api
```

``frontend/src/services/authService.js``

Fonctions API pour:

- ? inscription
- ? connexion
- ? profil connecte

``frontend/src/services/expenseService.js``

Fonctions API pour:

- ? lister les depenses
- ? creer
- ? modifier
- ? supprimer
- ? recuperer les stats
- ? recuperer les metadonnees

Si tu ajoutes un nouvel endpoint depense, ajoute une fonction ici.

``frontend/src/services/reportService.js``

Fonctions API pour:

- ? resume de rapport
- ? telechargement PDF

Le telechargement PDF cree un Blob, genere une URL temporaire et declenche un lien de telechargement.

``frontend/src/pages/DashboardPage.jsx``

Page du dashboard.

Elle affiche:

- ? bouton rapide pour ajouter une depense
- ? cartes de statistiques
- ? derniere depense

? categorie dominante

? graphiques

Elle appelle:

```
expenseService.stats()
```

Si tu veux ajouter un nouveau KPI dashboard, il faut:

1. ajouter le calcul dans `backend/expenses/views.py`;
2. afficher la donnee ici avec un `StatCard`.

`frontend/src/pages/ExpensesPage.jsx`

Page principale de gestion des depenses.

Elle orchestre:

- ? `ExpenseForm`
- ? `ExpenseFilters`
- ? `ExpenseTable`
- ? creation
- ? modification
- ? suppression
- ? rafraichissement de la liste

Apres un ajout reussi, le formulaire se vide pour eviter les doublons accidentels.

`frontend/src/pages/ReportsPage.jsx`

Page des rapports.

Elle permet:

- ? basculer entre journalier, hebdomadaire et mensuel
- ? voir le total
- ? voir les categories
- ? exporter en PDF

`frontend/src/pages/LoginPage.jsx`

Formulaire de connexion.

Utilise React Hook Form, puis appelle `useAuth().login()`.

`frontend/src/pages/RegisterPage.jsx`

Formulaire d'inscription.

Le mot de passe doit faire au moins 8 caracteres.

Les erreurs API sont affichees avec `getApiErrorMessage`.

``frontend/src/pages/ErrorPage.jsx``

Page affichee si une route React plante.

Elle evite l'ecran technique "Unexpected Application Error".

``frontend/src/layouts/AppLayout.jsx``

Layout principal apres connexion.

Contient:

? sidebar

? navigation

? header

? bouton logout

? zone `<Outlet />` pour afficher la page active

Pour ajouter une nouvelle page, ajoute aussi un lien dans le tableau nav.

``frontend/src/layouts/AuthLayout.jsx``

Layout des pages login/register.

``frontend/src/components/expenses/ExpenseForm.jsx``

Formulaire d'ajout/modification de depense.

Champs:

? montant obligatoire

? categorie obligatoire

? description optionnelle

? date obligatoire

? mode de paiement obligatoire

La description est volontairement optionnelle pour une saisie rapide.

``frontend/src/components/expenses/ExpenseFilters.jsx``

Filtres de recherche:

? recherche description

? categorie

? date debut

? date fin

? tri

Ces filtres sont envoyes directement a l'API via query params.

``frontend/src/components/expenses/ExpenseTable.jsx``

Tableau des dépenses.

Affiche:

? date

? description ou categorie si description vide

? categorie

? paiement

? montant

? actions modifier/supprimer

``frontend/src/components/dashboard/Charts.jsx``

Graphiques Recharts:

? pie chart categories

? evolution 30 jours

? histogramme mensuel

? depenses hebdomadaires

Si tu ajoutes de nouvelles stats, tu peux creer un nouveau composant graphique ici.

``frontend/src/components/dashboard/StatCard.jsx``

Carte reutilisable pour les KPI.

Utilisee dans le dashboard pour les totaux.

``frontend/src/components/common/``

Composants UI reutilisables:

? ``Button.jsx``

- ? ``Input.jsx``
- ? ``Select.jsx``
- ? ``Loader.jsx``
- ? ``EmptyState.jsx``

Important: Input et Select utilisent `forwardRef` pour fonctionner avec React Hook Form.

``frontend/src/hooks/useExpenses.js``

Hook custom pour charger les dépenses avec filtres.

Retourne:

- ? ``expenses``
- ? ``loading``
- ? ``refresh``

``frontend/src/utils/constants.js``

Labels et couleurs:

- ? categories
- ? moyens de paiement
- ? couleurs de graphiques

Si tu veux renommer une catégorie côté interface, c'est ici. Si tu veux ajouter une vraie catégorie en base, il faut aussi modifier le backend.

``frontend/src/utils/formatters.js``

Fonctions de formatage:

- ? devise EUR
- ? dates
- ? date du jour au format ISO

``frontend/src/utils/errors.js``

Transforme les erreurs API en messages lisibles.

Évite les messages vagues comme "Création impossible" quand le backend donne une raison précise.

Modifier le projet: chemin rapide

Ajouter une catégorie

Backend:

1. modifier `Expense.Category` dans `backend/expenses/models.py`;
2. creer une migration;
3. appliquer `python manage.py migrate`.

Frontend:

1. ajouter le label dans `frontend/src/utls/constants.js`;
2. ajouter une couleur dans `CATEGORY_COLORS`.

Ajouter un champ a une depense

Backend:

1. ajouter le champ dans `models.py`;
2. l'ajouter dans `ExpenseSerializer`;
3. creer et appliquer la migration.

Frontend:

1. ajouter le champ dans `ExpenseForm.jsx`;
2. l'afficher dans `ExpenseTable.jsx` si necessaire;
3. verifier que `ExpensesPage.jsx` envoie bien la valeur.

Ajouter une nouvelle page

1. creer un fichier dans `frontend/src/pages/`;
2. l'ajouter dans `frontend/src/routes/router.jsx`;
3. ajouter un lien dans `frontend/src/layouts/AppLayout.jsx`.

Ajouter un nouveau KPI dashboard

1. calculer la donnee dans `backend/expenses/views.py`, methode `stats`;
2. consommer cette donnee dans `DashboardPage.jsx`;
3. l'afficher avec `StatCard` ou un nouveau graphique.

Commandes utiles

Backend:

```
cd expense-tracker/backend
.venv\Scripts\activate
python manage.py runserver
python manage.py migrate
python manage.py check
```

Frontend:

```
cd expense-tracker/frontend
npm run dev
npm run lint
npm run build
```

Notes pour le deployment demain

Frontend Vercel:

? build command: `npm run build`

? output directory: `dist`

? variable: `VITE\_API\_URL=https://url-du-backend/api`

Backend Render/Railway:

? start command: `gunicorn config.wsgi:application`

? variables importantes:

- SECRET_KEY
- DEBUG=False
- ALLOWED_HOSTS
- CORS_ALLOWED_ORIGINS

Avant de deployer, il faudra choisir la base de données production. SQLite est parfait en développement, mais PostgreSQL est recommandé en production.

Idee pour la version mobile

Après le deployment web, l'évolution logique est une app mobile.

Approche recommandée:

- ? garder le backend Django REST tel quel;
- ? créer une app React Native avec Expo;
- ? réutiliser la logique API: auth, dépenses, stats, rapports;
- ? adapter les écrans: dashboard mobile, ajout rapide, liste, rapports.

Le backend actuel est déjà adapté pour servir une app mobile, car il expose une API JSON avec JWT.